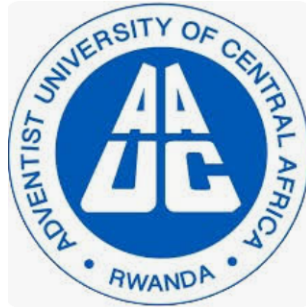




ADVENTIST UNIVERSITY OF CENTRAL AFRICA

**A Big Data Pipeline for Predicting Aircraft
Fuel Consumption Using Kernel Density Estimation
and Spark MLlib**

Big Data Essentials — Group Final Exam Project

Student Name	Student ID
Rubagumya Brave	101392
Rwagasana Felicien	101364
Uwizeye Christelle	101465

Program: MSc in Big Data Analytics

Course: Big Data Essentials

Lecturer: Dr. Kundan Kumar

GitHub: https://github.com/Rubagumyabrave/BigDataEssential_Final_Project

Date: September 18, 2026

1 Introduction

1.1 Motivation and Case Study

Fuel is one of the largest and most variable costs an airline faces, and small improvements in predicting fuel burn can support better planning decisions across a fleet. This project uses real historical flight data from a commercial airline as a case study, covering two aircraft types: a wide-body long-haul aircraft and a narrow-body aircraft flown in both passenger and freighter configurations. Because this data is confidential, it could not be shared directly between project members. Instead of working around this limitation with a substitute dataset, the project treated it as a design problem: fit a statistical model of the real data's shape, and generate new, realistic flights from that model rather than from the real data itself.

1.2 Research Question

Can synthetic flight data, generated from a density function fitted to real historical flights, support an accurate, fully deployed fuel-burn prediction system inside a real streaming big data architecture — without the system ever needing further access to the original confidential data?

1.3 Report Structure

Section 2 reviews related published work on aircraft fuel burn prediction. Section 3 describes the data and the cleaning and feature-selection work done on it. Section 4 explains the two modelling components: the synthetic data generator and the fuel-burn prediction model. Section 5 describes the system architecture that connects these pieces into a live pipeline. Section 6 presents the results, Section 7 discusses them, Section 8 states the project's honest limitations, and Section 9 concludes.

2 Related Work

A 2018 study (Yanto and Liem, *Transportation Research Part D*) built a fuel burn estimation model for 40 different aircraft types using only two predictors — mission distance and payload — fitted separately for each aircraft type. Their models achieved R^2 above 0.99 for every type studied. The same paper also tested pooling similar aircraft types together into shared models instead of keeping them separate, and found this substantially hurt accuracy, with R^2 falling to between 0.85 and 0.90. This result directly supports two choices made in this project: using distance and weight as the core predictors, and fitting separate models per aircraft group rather than one shared model.

This project's contribution is different in kind rather than in the modelling itself: it combines this style of fuel-burn modelling with a full, continuously running streaming big data architecture, and uses a fitted probability density, rather than the real data, as the source of new records feeding that architecture.

3 Data

3.1 Source and Structure

The working dataset was built from flight data recorder records supplied by the case-study airline, merged from three separate source files into one combined dataset before cleaning began.

3.2 Data Cleaning

Several real data quality issues were found and addressed before any modelling work began:

- **Unit conversion:** many numeric columns were stored as text with embedded units (for example, weight values written as “46901 kg”); these were parsed and converted to proper numeric types.
- **Duplicate rows:** the original single-source file contained 739 exact duplicate rows out of 1,689 (43.7%); the three-source merged file contained 802 duplicates out of 2,689 rows (29.8%). Both were removed.
- **A sensor artifact:** one weather-related column showed a physically impossible, repeated value on 524 of 1,401 rows (37%), traced to one of the three merged source files rather than to genuine sensor readings. Rather than use this contaminated data, the column was dropped from the feature set entirely.
- **Missing values:** remaining gaps were filled using a fallback chain — first a median value from the

same aircraft and route, then a fleet-level median — with rows dropped only when neither fallback produced a trustworthy value.

3.3 Feature Selection

Every numeric column was ranked by its correlation with total fuel burn. Columns that were algebraically derived from the target (for example, fuel quantity at landing, which combines with fuel burn by simple subtraction) were excluded as data leakage. Columns only knowable after a flight had already happened were also excluded from any model meant to score a flight in advance. The remaining candidate predictors were checked for multicollinearity using the variance inflation factor (VIF): despite a raw correlation of about 0.75 between distance and weight, VIF values came back at 1.69, 1.86, and 1.13 for distance, weight, and tailwind respectively — all comfortably under the conventional threshold of 5, confirming no problematic overlap between predictors.

3.4 Final Feature Set

The final model uses three predictors — distance, takeoff weight, and takeoff tailwind — to predict total fuel burn. After cleaning and feature selection, the final modelling dataset comprised 1,404 real historical flights, split unevenly across the two aircraft groups: 1,121 flights (79.8%) for the narrow-body group and 283 flights (20.2%) for the wide-body group. This imbalance reflects the airline's actual operational fleet mix rather than a sampling choice, and is one of the reasons each group is modelled separately rather than pooled together (Section 4.2). Figure 1 shows how the three predictors, plus fuel burn itself, are distributed for the two groups.

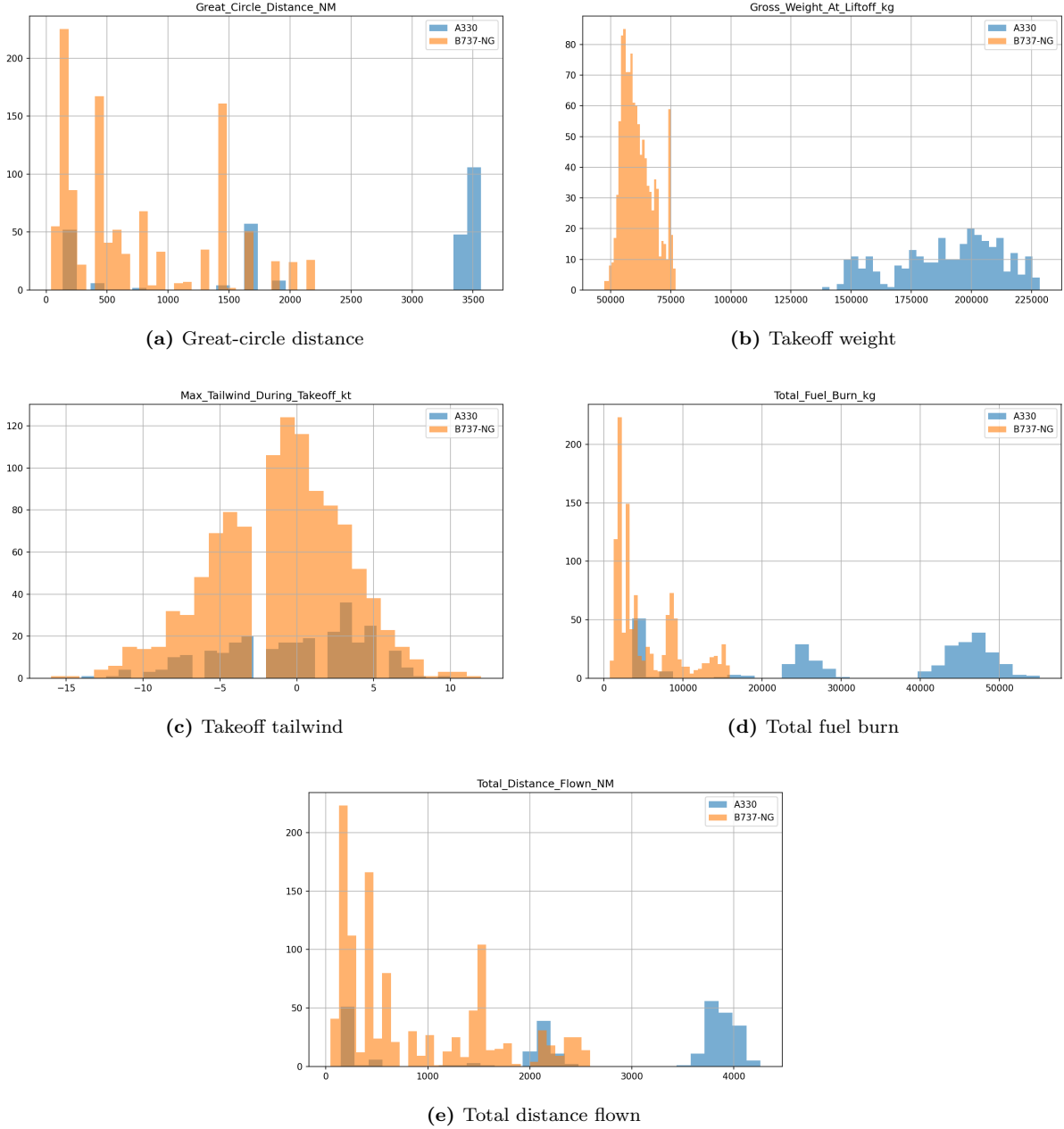


Figure 1: Distributions of great-circle distance, takeoff weight, takeoff tailwind, total fuel burn, and total distance flown, by aircraft group. Weight and distance are clearly separated between the two groups with almost no overlap, while tailwind is roughly symmetric around zero for both.

4 Methodology

4.1 Synthetic Data Generation

A joint kernel density estimate (KDE) was fitted separately for each aircraft group, over the log-transformed distance and weight together with tailwind. Fitting the three variables jointly, rather than sampling each independently, preserves the real correlation between them — most importantly, that longer flights tend to carry more weight. The smaller aircraft group, with only 47 real historical flights, was given a deliberately wider bandwidth (1.8 times the automatic setting) to avoid the density overfitting to individual data points. Every generated flight is checked against that group’s real historical range; draws that fall outside it are rejected and resampled, so the generator can never produce a physically implausible flight.

4.2 Fuel-Burn Model

A log-linear regression was trained with Spark MLlib to predict total fuel burn from distance, weight, and tailwind. Both distance and weight were log-transformed, which keeps their effect on fuel burn multiplicative and satisfies the assumption of constant variance in a linear model. The two narrow-body variants (passenger and freighter) were pooled into one shared model, since they share the same airframe and their weight ranges overlap almost completely. The wide-body group was kept as its own separate model, since its weight range does not overlap with the narrow-body group at all.

4.3 Train/Test Split

Rather than a random split, a temporal split was used: flights from earlier months formed the training set, and flights from later months formed the test set. This avoids the same route appearing in both halves, which a random split could allow, and gives a more honest picture of how the model performs on genuinely unseen flights.

5 System Architecture

The pipeline runs continuously: a generator produces synthetic flights, which pass through a REST API into Kafka, are automatically written into both a distributed file store and a relational database, are periodically scored by a trained model, and are shown on a live dashboard.

5.1 Rapid Data Generation

A continuous generator produces one synthetic flight every few seconds, at a rate that can be adjusted, and sends each one to a REST API for ingestion.

5.2 Kafka

A three-broker Kafka cluster hosts a single topic with three partitions and a replication factor of two. Producer messages are keyed by aircraft group, so that all flights from the same group consistently land on the same partition — verified directly by inspecting messages, which showed the same group mapping to the same partition every time. A separate custom Python consumer, running in its own named consumer group, was run as two instances at once to demonstrate Kafka reassigning partitions live when one instance stopped and restarted. This consumer commits its offsets automatically every five seconds, giving at-least-once delivery: a message could in principle be processed twice after a crash, but never silently dropped, which is the right trade-off for a consumer whose job is raising notifications rather than making irreversible changes.

5.3 Kafka Connect and Storage

Kafka Connect is configured to automatically sink every message into two places at once, with no manual copying step: a distributed file system for historical, analytical use, and a relational database for operational queries.

5.4 Batch Scoring

A Spark job reads the accumulated historical flights back out of the distributed file system, applies the trained model's coefficients to every record, and writes the resulting predictions into a dedicated table in the relational database. In one run of this job, 3,587 flights were scored this way.

5.5 Dashboard

A web dashboard, built with Django, reads directly from the relational database and presents an operational summary of raw flight traffic alongside the model's predictions and their distribution. A notable-flights panel, flagging unusually long or heavy flights using the same rule verified against the live Kafka stream, was also built into the dashboard as a planned enhancement beyond the base architecture.

6 Results

6.1 Model Coefficients and Elasticities

Table 1 shows the fitted coefficients for both models. Reading the distance coefficient as an elasticity: a 10% longer flight is predicted to burn about 7.1% more fuel for the narrow-body group, and about 6.7% more for the wide-body group. This is close to the distance elasticity found by an earlier, independently built model using a different statistical method on an earlier version of the dataset, which found values in the same 0.65–0.72 range. Two different models agreeing this closely is good evidence that the underlying relationship is real, not an artifact of one particular modelling choice. The tailwind coefficient is small and inconsistent in sign between the two groups, and is discussed honestly in the Limitations section.

	Narrow-body (pooled)	Wide-body
Intercept	−6.4264	−8.9322
log(distance) coefficient	0.7166	0.6803
log(weight) coefficient	0.9245	1.1476
Tailwind coefficient	−0.0031	+0.0015

Table 1: Fitted Spark MLlib log-linear regression coefficients, by aircraft group.

6.2 Model Evaluation

	Narrow-body (pooled)	Wide-body
Test-set flights	290	94
RMSE (kg)	519	1,014
MAE (kg)	350	791

Table 2: Model evaluation on held-out test data (later months).

6.3 Insights from the Stored Data

A Spark job reading the historical data from the distributed file store produced several findings:

- **Distance range:** the wide-body group’s flights ranged from 146 to 4,242 nautical miles (a spread of 4,096 NM) — far wider than the narrow-body group’s 47–2,563 NM (spread of 2,516 NM), reflecting a more varied mix of short and long routes.
- **Wind conditions:** the freighter variant’s flights were markedly more headwind-skewed (849 headwind vs. 257 tailwind, roughly 3.3 to 1) than either the passenger narrow-body flights (571 vs. 447) or the wide-body flights (564 vs. 515), which were closer to balanced.
- **Heaviest recorded flight per group:** 227,717 kg over 4,079 NM for the wide-body group, and 76,727 kg over 2,012 NM for the narrow-body group — both at the top end of each group’s fitted range, as expected.

6.4 Fuel Burn vs. Distance

Figure 2 plots every real historical flight’s distance against its actual fuel burn, colored by aircraft group. The relationship is visibly strong and consistent within each group, with fuel burn rising steadily as distance increases — direct visual confirmation of the elasticity reported in Section 6.1. The distinct clusters visible within the narrow-body group reflect that fleet’s mixed short, medium, and longer route network.

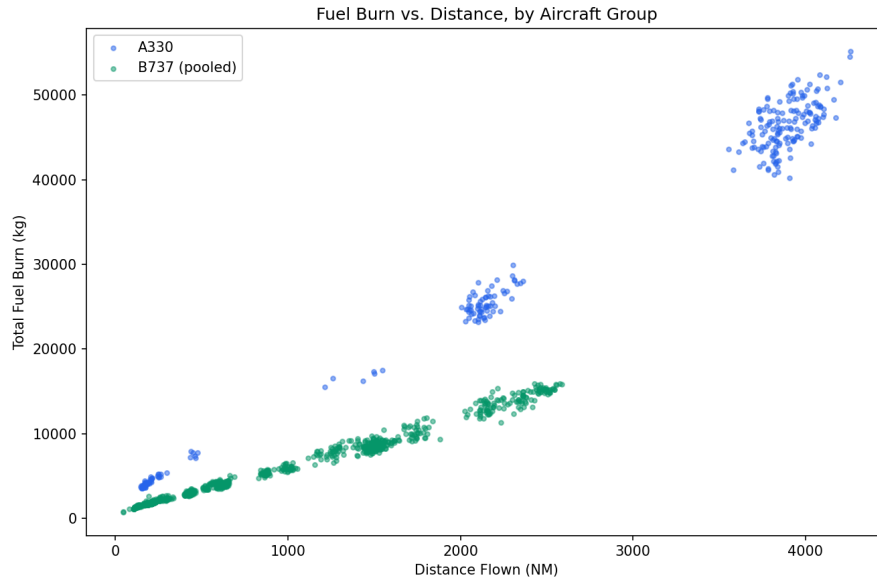


Figure 2: Total fuel burn against distance flown, by aircraft group, using the real historical dataset.

6.5 Model Validation

Figure 3 tests the trained model directly against real data: every real historical flight's known distance, weight, and tailwind were passed through the model's own fitted equation, and the resulting prediction was compared against that flight's actual recorded fuel burn. The left panel overlays predicted values directly on the actual values against distance; the right panel plots predicted fuel burn against actual fuel burn, with a dashed line marking a perfect prediction. Points cluster tightly along that line across the full range for both aircraft groups, giving direct visual support to the RMSE and MAE figures reported in Section 6.2.

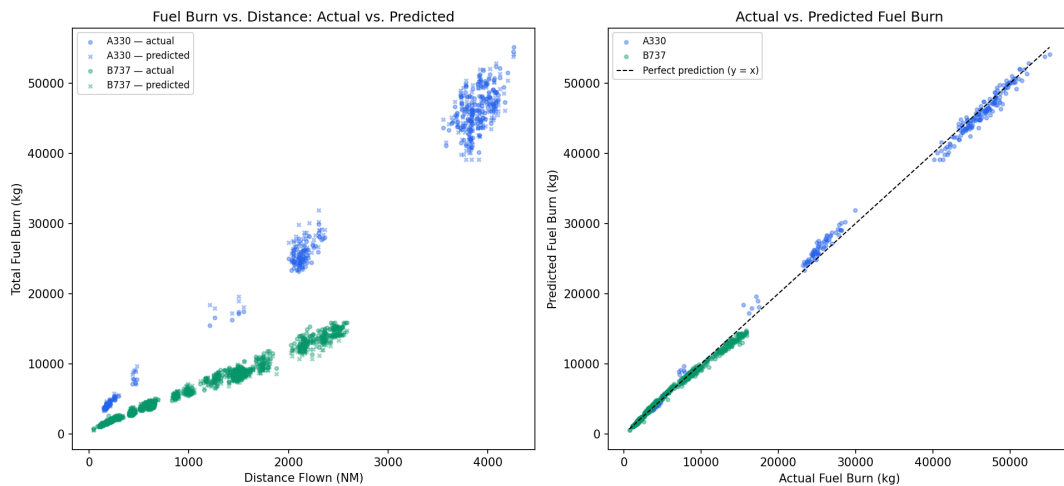


Figure 3: Actual vs. model-predicted fuel burn for every real historical flight, by aircraft group.

6.6 Notable-Flight Detection

A rule-based check flags any flight whose distance or weight falls in the top 10% of its aircraft group's real historical range. This was verified directly against the live Kafka stream: the custom consumer correctly identified genuine long-haul and heavy flights in real time as they were generated. The same rule was also built into the dashboard as a planned enhancement beyond the base architecture.

7 Discussion

7.1 Comparison Against Published Benchmarks

The published benchmark reviewed in Section 2 achieved R^2 above 0.99 using the same core predictors (distance and payload) on a much larger, higher-fidelity dataset. This project's results, while less precise, follow the same structural pattern and support the same central conclusion: distance and weight, modelled per aircraft type rather than pooled across types, are strong, sufficient predictors of fuel burn.

7.2 Consistency Between Independent Models

The distance elasticity found by this project's Spark MLlib model closely matched an earlier, separately fitted model using a different statistical approach. Agreement between two independently built models is stronger evidence than either result alone.

7.3 Engineering Challenges

Several genuine infrastructure problems were diagnosed and resolved during the project. Early attempts to write into the distributed file store failed because its storage directories were owned by a different user than the one the write process ran as; this was fixed by correcting directory ownership before any data flowed. Later, reads from the distributed file store failed intermittently when run from outside its own network — traced to unreliable networking between the host machine and the storage containers, and resolved by running the analysis job inside the same container network as the storage itself, rather than reaching across that network boundary.

8 Limitations

1. The smaller aircraft group had only 47 real historical flights to fit a density on; a wider bandwidth was used to reduce overfitting, but this remains a genuinely thinner sample than the other group.
2. The tailwind coefficient is small and flips sign between the two groups. This is an honest limitation of the available data: only takeoff and descent-phase wind measurements were available, not cruise-level headwind, which is the dominant weather influence on real fuel burn.
3. The distributed storage setup used a single node with no data replication, meaning no fault tolerance and reduced reliability under heavy concurrent read and write load.
4. The deployed model scores flights in scheduled batches rather than continuously as each one arrives. The project's own assignment brief explicitly allows either approach; batch scoring was chosen as the simpler, more standard option for this kind of historical analysis.

9 Conclusion

This project built a complete, working big data pipeline that generates realistic synthetic flight data from a density fitted to real, confidential flight records, and uses that synthetic stream to demonstrate every stage of a modern streaming architecture: ingestion, partitioned messaging, dual storage, batch-scored machine learning, and a live dashboard. The fuel-burn model's central finding — that fuel burn rises with distance at a consistent, predictable rate of roughly 7% per 10% increase in distance — held steady across two independently built models and is consistent with published literature on the same problem. The project also surfaced and resolved real infrastructure challenges along the way, and is transparent about its remaining limitations, particularly around sample size for one aircraft group and the weak signal from the available wind data.

10 References

1. Yanto, J., & Liem, R. P. (2018). Aircraft fuel burn performance study: a data-enhanced modeling approach. *Transportation Research Part D: Transport and Environment*, 65, 574–595.